

비동기 처리

async/await

async/await 문법



Promise의 단점

```
function fetchData(url) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve(`Data from ${url}`);  
    }, 1000);  
  });  
}
```

```
fetchData('https://api.example.com/data1')  
  .then((data1) => {  
    console.log(data1);  
    return fetchData('https://api.example.com/data2');  
  })  
  .then((data2) => {  
    console.log(data2);  
    return fetchData('https://api.example.com/data3');  
  })  
  .then((data3) => {  
    console.log(data3);  
    return fetchData('https://api.example.com/data4');  
  })  
  .then((data4) => {  
    console.log(data4);  
    // 계속해서 체이닝...  
  })  
  .catch((error) => {  
    console.error('Error:', error);  
  });
```

Promise를 이용하여 비동기를 처리하는 경우,
체이닝이 계속되면 가독성이 좋지 않은 코드가 될 수 있다.



Promise의 단점

```
function fetchData(url) {  
  return new Promise((resolve, reject) => {  
    setTimeout(() => {  
      resolve(`Data from ${url}`);  
    }, 1000);  
  });  
}
```

```
async function fetchAllData() {  
  try {  
    const data1 = await fetchData('https://api.example.com/data1');  
    console.log(data1);  
  
    const data2 = await fetchData('https://api.example.com/data2');  
    console.log(data2);  
  
    const data3 = await fetchData('https://api.example.com/data3');  
    console.log(data3);  
  
    const data4 = await fetchData('https://api.example.com/data4');  
    console.log(data4);  
  
    // 계속해서 순차적으로 호출...  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}  
  
fetchAllData();
```

ES8부터 도입된 async/await 문법을 사용하면,
비동기 처리를 훨씬 깔끔하게 작성할 수 있다.



async/await

```
async function fetchData() {  
  try {  
    const data = await fetchData('https://api.example.com/data1');  
    console.log(data);  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}  
  
fetchData();
```

async/await을 이용해 비동기 처리를 동기 처리하듯이 작성할 수 있다.

```
async function fetchData() {  
  try {  
    const data = await fetchData('https://api.example.com/data1');  
    console.log(data);  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}  
  
fetchData();
```

‘await’ 키워드를 사용하면, 해당 비동기 처리가 완료될 때까지 기다린 후,
resolve 된 값을 반환한다.

```
async function fetchData() {  
  try {  
    const data = await fetchData('https://api.example.com/data1');  
    console.log(data);  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}  
  
fetchData();
```

‘await’ 키워드는 무조건 ‘async’ 함수 내부에서만 사용할 수 있다.


```
// 일반 함수
async function square(n) { return n * n; }
square(3).then(result => console.log(result)); // 9

// 화살표 함수
const double = async (n) => { return n * 2; }
double(5).then(result => console.log(result)); // 10
```

async 함수는 항상 반환값을 resolve하는 Promise를 반환한다.

예외 처리



```
const jsonString = '{"name": "John", "age": 30'; // JSON 형식에 맞지 않음
JSON.parse(jsonString); // SyntaxError
```

모든 프로그램은 에러가 발생할 가능성이 존재하며, 발생시 프로그램은 종료된다.

종료를 방지하기 위해 예외 상황에 최대한 대응하는 코드를 작성해야 한다.



async/await

```
async function fetchData() {  
  try {  
    const data = await fetchData('https://api.example.com/data1');  
    console.log(data);  
  } catch (error) {  
    console.error('Error:', error);  
  }  
}  
  
fetchData();
```

async/await 사용 시에는 **try ... catch ... finally** 문을 사용하여 예외 처리를 진행한다.



try ... catch 예외 처리

```
const jsonString = '{"name": "John", "age": 30}'; // JSON 형식에 맞지 않음

try {
  JSON.parse(jsonString); // SyntaxError
} catch (err) {
  console.log(err);
}
```

기본적으로 try ... catch 문을 이용해 예외를 처리할 수 있다.

(try만 단독으로 사용할 수는 없다.)



try ... catch ... finally 예외 처리

```
const jsonString = '{"name": "John", "age": 30'; // JSON 형식에 맞지 않음

try {
  JSON.parse(jsonString); // SyntaxError
} catch (err) {
  console.log(err);
} finally {
  console.log("JSON 처리 완료");
}
```

finally 문을 추가해 에러 여부와 상관없이 실행되는 코드를 작성할 수 있다.